0097. Interleaving String

Category: 2-D Dynamic Programming | LeetCode: https://leetcode.com/problems

Problem

- Is s3 formed by interleaving s1 and s2 (keeping each string's internal order)?

Intuition

- Dp[i][j] = can the first i chars of s1 and the first j chars of s2 form the first i+j chars of s3?
- The last char of that prefix came from s1 (dp[i-1][j] && s1[i-1] == s3[i+j-1]) or from s2 (dp[i][j-1] && s2[j-1] == s3[i+j-1]).

Brute Force

- Recursion choosing s1 or s2 at each step: O(2^(m+n)).

Optimal Approach

- If m + n != |s3| return false.
- Fill dp with dp[0][0] = true using the rule above, one row at a time (1-D array).

Worked Example

```
s1 "aabcc", s2 "dbbca", s3 "aadbbcbcac" -> true.

s1 "aa"   -> aa
s2 "dbbc" -> aadbbc
s1 "bc"   -> aadbbcbc
s2 "a"    -> aadbbcbca
s1 "c"    -> aadbbcbcac   == s3, so dp[5][5] = true
```

Java Solution

```java
public class InterleavingString {
    public boolean isInterleave(String s1, String s2, String s3) {
        int m = s1.length(), n = s2.length();
        if (m + n != s3.length()) return false;
        boolean[] dp = new boolean[n + 1];
        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0 && j == 0) { dp[0] = true; continue; }
                char c = s3.charAt(i + j - 1);
                boolean fromS1 = i > 0 && dp[j] && s1.charAt(i - 1) == c;
                boolean fromS2 = j > 0 && dp[j - 1] && s2.charAt(j - 1) == c;
                dp[j] = fromS1 || fromS2;
            }
        }
        return dp[n];
    }
}
```

Complexity

- Time: O(m n). Space: O(n).

Pitfalls

- The length check up front avoids index errors.